

MiniHTTPd

Francis Bravo

MiniHTTPd

Servidor HTTP/1.1 minimo en C con epoll — Francis Bravo

El proyecto consistio en desarrollar desde cero un servidor web HTTP/1.1 en lenguaje C bajo Linux (Ubuntu 26.04 sobre WSL2), sin recurrir a ninguna biblioteca HTTP externa. La implementacion final consta de aproximadamente 1500 lineas distribuidas en cinco modulos (main, server, http, mime, files), y soporta servir archivos estaticos del directorio `www/` con tipos MIME automaticos, manejo concurrente de multiples clientes mediante epoll, conexiones persistentes con un timeout de inactividad de 30 segundos, y proteccion contra el ataque de Directory Traversal. El servidor escucha en el puerto 8080 y responde correctamente con los codigos 200, 400, 403, 404 y 405 segun la situacion.

La parte mas dificil del proyecto fue practicamente toda, porque era la primera vez que me enfrentaba a la programacion en bajo nivel con sockets, al modelo event-driven con epoll y al parsing manual del protocolo HTTP. Conceptos como el “3-way handshake” de TCP, las llamadas al sistema `socket()`, `bind()`, `listen()`, `accept()`, o la diferencia entre lectura bloqueante y no bloqueante con `fcntl(O_NONBLOCK)`, eran completamente nuevos para mi. Tener que pensar en terminos de descriptores de archivo en lugar de objetos de alto nivel implico un cambio de mentalidad importante. Esta curva de aprendizaje sostenida hizo que cada paso del proyecto exigiera comprender primero el porque antes de escribir el codigo.

Lo que mas me sorprendio durante las pruebas fue observar el comportamiento real del navegador al acceder a la pagina servida por mi propio servidor. Al cargar `http://localhost:8080/`, Chrome no abrio una sola conexion sino varias en paralelo (`fd=5`, `fd=6`, `fd=7` simultaneamente en los logs), reutilizando algunas para descargar el HTML, el CSS, la imagen PNG y el favicon. Verificar que mi servidor manejara correctamente todas esas conexiones concurrentes gracias al bucle de epoll fue una validacion practica del modelo event-driven que hasta ese momento solo entendia en teoria. Tambien me sorprendio comprobar que el navegador asume keep-alive por defecto en HTTP/1.1 y que mi servidor debia respetar explicitamente la cabecera `Connection` del cliente.

Entre los bugs especificos que enfrente, el mas memorable fue uno donde habia escrito `if (*line_end == "\0")` usando comillas dobles en lugar de comillas simples. En C, las comillas dobles definen un string literal (un puntero), mientras que las comillas simples definen un caracter. Al comparar un caracter (`*line_end`) contra un puntero (`"\0"`), el compilador genero el warning **comparison between pointer and integer**, y la comparacion nunca era verdadera. Identificar este detalle ortografico minimo, pero con un impacto funcional total, fue una leccion sobre la importancia de cada caracter en C. Otro bug significativo fue una funcion auxiliar de “trim” cuya condicion logica estaba invertida, lo cual provocaba que los valores de las cabeceras HTTP quedaran vacios; lo encontre agregando trazas `printf` paso a paso hasta aislar el origen.

En cuanto a decisiones de diseno, opte por el modo Level-Triggered de epoll en lugar de Edge-Triggered porque simplifica la logica de lectura a costa de una syscall extra por mensaje, lo cual es aceptable para la escala de este proyecto. Para el timeout de keep-alive implemente un mecanismo de barrido (sweep)

que recorre una tabla `last_activity[]` indexada por descriptor de archivo cada vez que `epoll_wait` retorna; este enfoque es $O(\text{MAX_CLIENTS})$ por segundo y suficiente para los 1024 clientes simultaneos que soporto. Para la seguridad utilice `realpath()` para resolver la ruta absoluta del archivo solicitado y validar que el prefijo coincida exactamente con el directorio `www/` resuelto al arrancar; esto bloquea ataques como `GET ../../../../etc/passwd` retornando 403 Forbidden.

Comparacion con Apache y Nginx. Apache en su modo prefork usa un proceso por conexion, lo cual es simple pero limita la escalabilidad por el costo de memoria por proceso. Nginx, en cambio, emplea un modelo event-driven con multiples workers (uno por core) que multiplexan conexiones con `epoll` en modo Edge-Triggered y no bloqueante; este diseno le permite atender decenas de miles de conexiones simultaneas (problema C10K) con consumo de memoria estable. MiniHTTPd se asemeja conceptualmente a un solo worker de Nginx: un unico hilo, `epoll` como multiplexor, y conexiones no bloqueantes. La diferencia clave es que no implementa el fan-out a multiples workers para aprovechar SMP, ni usa optimizaciones avanzadas como `sendfile()` para evitar copias entre espacio de usuario y kernel. Como mejoras futuras, considero extender el servidor con multiples workers via `fork()`, migrar a Edge-Triggered, integrar `sendfile()` para archivos grandes, y soportar HTTP/2 multiplexado.

Declaracion de ayuda externa. Para el desarrollo de este proyecto utilice asistencia de inteligencia artificial (Claude de Anthropic) como tutor interactivo. La IA me guio paso a paso explicando conceptos teoricos (sockets, `epoll`, parsing HTTP, directory traversal, semantica de RFC 7230), me sugirio la estructura modular del codigo, y me ayudo a diagnosticar bugs mediante depuracion conjunta con `printf` de trazas. Sin embargo, todas las decisiones finales, la redaccion de los comentarios, la adaptacion del codigo a mis apuntes personales, y las pruebas de validacion las realice yo. La IA actuo como un tutor tecnico, no como autor del codigo. Tambien consulte las RFC 7230 y 9112 para entender la sintaxis y semantica formal del protocolo HTTP/1.1.

Repositorio GitHub: https://github.com/Francis1918/Servidor_Mini_HTTPd